Merged Files List

1. api_developer.md (2.4 KB)
2. product_and_data_analytics_analyst.md (2.6 KB)
3. mobile_app_developer.md (2.7 KB)
4. big_data_engineer.md (2.7 KB)
5. cloud_engineer.md (3.8 KB)
6. application_security_engineer.md (3.5 KB)
7. penetration_tester.md (3.5 KB)
8. security_engineer.md (3.5 KB)
9. cybersecurity_analyst.md (3.3 KB)
10. ai_engineer.md (3.3 KB)
11. devops_engineer.md (3.2 KB)
12. cloud_solutions_architect.md (3.2 KB)
13. generative_ai_engineer.md (3.2 KB)
14. backend_developer.md (3.2 KB)
15. java_developer.md (3.1 KB)
16. ai_research_engineer.md (3.1 KB)
17. software_engineer.md (3.1 KB)
18. nlp_engineer.md (3.1 KB)
19. machine_learning_engineer.md (3.1 KB)
20. bi_and_reporting_analyst.md (3.1 KB)
21. site_reliability_engineer.md (3 KB)
22. soc_analyst.md (2.9 KB)
23. frontend_developer.md (2.9 KB)
24. data_engineer.md (2.9 KB)
25. platform_engineer.md (2.9 KB)
26. python_developer.md (2.9 KB)
27. computer_vision_engineer.md (2.8 KB)
28. full_stack_developer (1).md (2.8 KB)
29. data_analyst.md (2.8 KB)
30. data_scientist.md (2.8 KB)
31. analytics_engineer.md (2.8 KB)

1. api_developer.md

```
# API Developer

## Role guide
An API developer designs, builds, documents and secures the interfaces that let software systems talk to each other, such as a mobile

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: how HTTP works (methods, headers, status codes), client and server, JSON, how databases store data, basic security
- Programming: one backend language (Python, Java or JavaScript) and SQL.
- Development environment: Git and GitHub, Postman or a similar API testing tool, VS Code or an IDE, the command line.
- Switching from another field: testers and frontend developers already know part of this. Start by calling public APIs, then build

### 2. Core and technical skill progression
- Level 1, fundamentals: REST principles, routing, CRUD operations, request validation, correct status codes, JSON responses, and tes
- Level 2, core competencies: authentication and authorization (API keys, tokens, OAuth basics), pagination and filtering, error hand
- Level 3, advanced: API gateways, communication between services (microservices), alternatives such as GraphQL and gRPC, caching, ba
- Rough guide to experience: Level 1 matches 0 to 2 years of work, Level 2 about 2 to 5 years, Level 3 about 5 years and beyond.

### 3. Practical tooling and ecosystem
- Frameworks: FastAPI, Flask or Django REST Framework; Spring Boot; Express.
- Documentation and testing: OpenAPI or Swagger, Postman, automated test frameworks (pytest, JUnit, Jest).
- Databases: PostgreSQL or MySQL, Redis for caching.
- Infrastructure: API gateways, Docker, CI pipelines, cloud platforms.

### 4. Hands-on projects and portfolio milestones
- Beginner: a notes or to-do API, and a script that consumes a public API.
- Intermediate: a blog or e-commerce API with login, roles, validation, documentation and tests.
- Advanced: an API with rate limiting, versioning, caching and monitoring, load tested and deployed with CI.
```

2. product_and_data_analytics_analyst.md

```
# Product Analyst / Product and Data Analytics Roles

## Role guide
A product analyst studies how users behave in a product to guide product decisions. The work includes exploratory analysis, funnel a

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: product metrics (funnels, retention, cohorts, engagement, lifetime value), statistics (hypothesis testing, confiden
- Programming: SQL, then Python or R.
- Development environment: Excel or Google Sheets, Jupyter Notebook, a BI tool, and Git basics.
- Switching from another field: marketing, operations or business analysts can add SQL, statistics and experimentation.

### 2. Core and technical skill progression
- Level 1, fundamentals: SQL, Excel, basic data visualization, defining metrics, attention to detail.
- Level 2, core competencies: advanced SQL, funnel analysis (where users drop off), cohort retention analysis (whether users stay), A
- Level 3, advanced: experimentation strategy and quasi-experiments, segmentation, regression and causal inference, setting standard
- A common trap: testing many metrics at once increases the chance of a false positive, so decide the main metric in advance.

### 3. Practical tooling and ecosystem
- SQL and data warehouses for user-level product data.
- Python or R for statistics and analysis.
- BI tools: Tableau, Looker, Power BI.
- Experimentation and product analytics tools (the specific platform depends on the company).

### 4. Hands-on projects and portfolio milestones
- Beginner: a funnel analysis in SQL on a public e-commerce dataset.
- Intermediate: a cohort retention analysis and a retention dashboard, and an A/B test analysis with a written recommendation.
- Advanced: an experiment design document with sample size and success metrics, followed by a full analysis and a decision memo for a
```

3. mobile_app_developer.md

```
# Mobile Application Developer (Android, iOS and cross-platform)

## Role guide
A mobile app developer designs, builds and maintains apps for phones and tablets. The first decision is the platform: Android, iOS, c

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: programming basics, object-oriented ideas, how apps use screens, navigation and data, how APIs and JSON work, basic
- Programming: choose a platform first. Android: Kotlin (the language Google recommends for Android today). iOS: Swift. Cross-platfo
- Development environment: Git and GitHub, Android Studio for Android, Xcode for iOS (which generally needs a Mac), emulators and a r
- Switching from another field: JavaScript developers can move to React Native, and others start with one platform and small apps.

### 2. Core and technical skill progression
- Level 1, fundamentals: the language, building screens, layouts, navigation, handling user input, and small apps.
- Level 2, core competencies: modern UI toolkits (Jetpack Compose or SwiftUI, Flutter widgets), state management, calling APIs, local
- Level 3, advanced: app architecture for large apps, performance and memory tuning, offline sync, release management, security, CI/C
- Rough guide to experience: Level 1 matches 0 to 2 years of work, Level 2 about 2 to 5 years, and Level 3 about 5 years and beyond.

### 3. Practical tooling and ecosystem
- Frameworks: Jetpack Compose, SwiftUI, Flutter, React Native; Kotlin Multiplatform as another option.
- Data: SQLite or Room, Firebase, REST APIs.
- Release and quality: app signing and store requirements, crash reporting (for example Firebase Crashlytics), CI tools such as GitH
Choose native when you want one platform in depth, and cross-platform when you want to reach both quickly.

### 4. Hands-on projects and portfolio milestones
- Beginner: calculator, to-do list, a weather app that calls an API.
- Intermediate: a quiz or news app, a notes app with local storage and login, published or with a signed build.
- Advanced: a chat or e-commerce app with architecture, tests, offline support, analytics and a release pipeline.
```

4. big_data_engineer.md

```
# Big Data Engineer

## Role guide
A big data engineer builds batch and streaming pipelines that process very large datasets across clusters of machines, using tools su

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: the characteristics of big data (volume, velocity, variety), distributed systems basics, and big data file formats s
- Programming: Python, SQL, Linux and basic Java. Hadoop and Spark components require the Java Development Kit. SQL stays important h
- Development environment: Git, the command line, and a local or cloud cluster for practice.
- Switching from another field: close any gaps in SQL, Python and Linux before attempting Hadoop or Spark.

### 2. Core and technical skill progression
- Level 1, fundamentals: strong SQL, Python (Scala or Java for some roles), Linux, and big data concepts and file formats.
- Level 2, core competencies: the Hadoop ecosystem (HDFS, YARN, MapReduce, Hive, HBase), Apache Spark (RDDs, DataFrames, Spark SQL, h
- Level 3, advanced: Kafka and stream processing (Spark Streaming, Flink), cloud platforms and lakehouse concepts, performance optim
- Learning each stage through coding and projects is advised, not only theory.

### 3. Practical tooling and ecosystem
- Processing: Hadoop, Hive, HBase, Spark, Flink.
- Messaging and streaming: Kafka, Kafka Connect.
- Languages: Scala, Python, Java.
- Databases: NoSQL stores such as Cassandra and MongoDB, and SQL warehouses such as BigQuery.
- Cloud and orchestration: AWS, GCP or Azure, Airflow.
- Certifications named in guides: Google Cloud Professional Data Engineer, Azure Data Engineer Associate.

### 4. Hands-on projects and portfolio milestones
- Beginner: SQL and Hive queries on a large public dataset.
- Intermediate: a Spark batch job that cleans and aggregates a large dataset, written in PySpark or Scala.
- Advanced: a real-time pipeline with Kafka and Spark Streaming, or a lakehouse design on a cloud provider, with performance notes.
```

5. cloud_engineer.md

```
# Cloud Engineer

## Role guide
A cloud engineer designs, deploys and maintains the infrastructure that applications run on in the cloud: compute, storage, networki

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: cloud computing basics (IaaS, PaaS, SaaS, public, private and hybrid cloud), networking (IP addressing, subnetting,
- Programming: Bash and Python scripting for automation. Employers want practical skills more than advanced programming.
- Development environment: Git and GitHub, the terminal and SSH, VS Code, and a free-tier cloud account for practice.
- Switching from another field: people from IT support, system administration, networking or development can move in. With 6 to 9 mo

### 2. Core and technical skill progression
- Level 1, fundamentals: choose one cloud platform and learn it deeply. AWS has the most job openings, Azure is strong in enterprise
- Level 2, core competencies: Infrastructure as Code (Terraform is cloud-agnostic and widely valued, while CloudFormation and Bicep
- Level 3, advanced: Kubernetes (EKS, AKS, GKE), cloud security and DevSecOps, designing for high availability, cost and performance
- Certifications support learning but do not replace understanding Linux, networking and systems thinking. A strong cloud engineer c

### 3. Practical tooling and ecosystem
- Platforms: AWS (EC2, S3, IAM, VPC, RDS, Lambda), Azure (virtual machines, virtual networks, Functions), GCP.
- Automation: Terraform, Ansible, CloudFormation.
- Containers and delivery: Docker, Kubernetes, GitHub Actions, Jenkins.
- Monitoring: CloudWatch, Azure Monitor, Prometheus, Grafana.
- Certifications often named: AWS Solutions Architect Associate, Azure Administrator (AZ-104), Google Associate Cloud Engineer.

### 4. Hands-on projects and portfolio milestones
- Beginner: host a static website on cloud storage, launch a virtual machine and connect over SSH.
- Intermediate: provision a network, a server and a storage bucket with Terraform and keep the state in remote storage, and containe
- Advanced: a CI/CD pipeline that deploys to Kubernetes with monitoring and alerts, and a secure multi-tier architecture with least-
```

6. application_security_engineer.md

```
# Application Security Engineer

## Role guide
An application security (AppSec) engineer secures the software itself across its whole lifecycle. The work includes manual and automa

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: how web applications and APIs work (HTTP, sessions, authentication), the software development lifecycle, and the co
- Programming: proficiency in at least one language (Python, JavaScript or TypeScript, Java), enough to read other people's code and
- Development environment: Git and GitHub, an IDE, Docker, and a CI system such as GitHub Actions.
- Switching from another field: developers learn the security side, and penetration testers and DevOps engineers learn code review a

### 2. Core and technical skill progression
- Level 1, fundamentals: the OWASP Top 10 and common vulnerabilities (such as injection and broken access control), secure coding ba
- Level 2, core competencies: static analysis (SAST), dynamic analysis (DAST) and software composition analysis (SCA) of dependencie
- Level 3, advanced: integrating scanning tools into CI/CD with automated quality gates and little friction for developers, containe

### 3. Practical tooling and ecosystem
- Code and dependency scanning: Semgrep, CodeQL, Snyk, Dependabot, Checkmarx, GitHub Advanced Security.
- Dynamic testing: OWASP ZAP, Burp Suite.
- Supply chain: OWASP Dependency-Track and SBOM tools.
- Pipelines: GitHub Actions, GitLab CI, Jenkins.
- Languages: Python, TypeScript or JavaScript.

### 4. Hands-on projects and portfolio milestones
- Beginner: find and fix vulnerabilities in an intentionally vulnerable web app.
- Intermediate: add code scanning and dependency scanning to a GitHub Actions pipeline and triage the results, and write a threat mo
- Advanced: build a DevSecOps pipeline that combines SAST, SCA, secrets scanning and DAST in staging with automated gates, and write
```

7. penetration_tester.md

```
# Penetration Tester (Ethical Hacker)

## Role guide
A penetration tester is an ethical hacker who is authorized to attack systems, applications and networks in order to find weaknesses

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: networking (you cannot test what you do not understand), Linux (non-negotiable for this role), operating system secu
- Programming: Python for scripting and automation, plus Bash.
- Development environment: Kali Linux, virtual machines, and legal practice labs such as TryHackMe and Hack The Box. Only test syste
- Switching from another field: SOC analysts, system administrators, network engineers and developers move across most easily.

### 2. Core and technical skill progression
- Level 1, fundamentals: networking and Linux, security basics, an entry certification such as CompTIA Security+, and guided beginne
- Level 2, core competencies: web application security (the OWASP Top 10, Burp Suite, OWASP ZAP), network scanning with Nmap, exploit
- Level 3, advanced: the OSCP, a hands-on certification that is widely respected because it is a practical exam and not multiple cho

### 3. Practical tooling and ecosystem
- Core tools: Kali Linux, Nmap, Burp Suite (it appears in a large share of job postings), Metasploit, Wireshark, OWASP ZAP, password
- Practice platforms: TryHackMe, Hack The Box, deliberately vulnerable web apps.
- Skills around the tools: scripting in Python, note-taking and clear reporting.

### 4. Hands-on projects and portfolio milestones
- Beginner: complete guided lab paths and keep notes on every machine.
- Intermediate: solve CTF challenges and write up each one, and test an intentionally vulnerable web application in your own lab.
- Advanced: run a full test of a multi-machine lab environment and write a professional report with an executive summary, findings w
```

8. security_engineer.md

```
# Security Engineer

## Role guide
A security engineer designs, builds and improves the security controls of an organization's IT, application and cloud environments. T

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: networking (TCP/IP, DNS, HTTP and HTTPS, routing), operating system internals, file permissions and hardening, ident
- Programming: Python for automation, APIs and log analysis, plus Bash and PowerShell.
- Development environment: Linux, Git and GitHub, virtual machines and a home lab where you can safely practise.
- Switching from another field: security analysts, system administrators, network engineers and developers are the usual routes. Sec

### 2. Core and technical skill progression
- Level 1, fundamentals: Linux and Windows administration, networking, scripting, security basics and an entry-level certification su
- Level 2, core competencies: network security (firewalls, intrusion detection and prevention, VPNs, segmentation), identity and acce
- Level 3, advanced: detection engineering (building alerts, improving log data quality and reducing noise), cloud guardrails define

### 3. Practical tooling and ecosystem
- Detection and response: SIEM platforms, EDR tools, cloud threat detection services (for example GuardDuty and CloudTrail on AWS, Se
- Cloud and infrastructure: AWS, Azure or GCP security services, Terraform, Kubernetes, cloud security posture management (CSPM) too
- Languages: Python, Bash, PowerShell.

### 4. Hands-on projects and portfolio milestones
- Beginner: harden a Linux virtual machine against a checklist you wrote.
- Intermediate: build a small detection pipeline in a lab (collect logs into an open-source SIEM and write alerts), and set up least-
- Advanced: define cloud security guardrails as code, add automated checks for misconfigurations, and write a security architecture
```

9. cybersecurity_analyst.md

```
# Cybersecurity Analyst

## Role guide
A cybersecurity analyst protects an organization's networks and systems from threats. The day-to-day work is often less dramatic thar

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: how Windows and Linux work, networking (TCP/IP, DNS, firewalls, VPNs, subnetting, routers and switches), and securit
- Programming: Python and Bash for automating tasks and analyzing data. You do not need to be a developer, but the role leans techni
- Development environment: a home lab with Windows and Linux virtual machines, the Linux command line, Git, and a text editor.
- Switching from another field: IT support, system administration and networking are the most common routes. People with no IT backgr

### 2. Core and technical skill progression
- Level 1, fundamentals: IT and networking basics, operating systems, security concepts, and reading logs such as Windows event logs
- Level 2, core competencies: SIEM tools (Splunk, QRadar, Microsoft Sentinel, and open-source ones such as Wazuh or Security Onion),
- Level 3, advanced: tuning detections and reducing false positives, threat intelligence, leading incident response, digital forensi

### 3. Practical tooling and ecosystem
- SIEM: Splunk, IBM QRadar, Microsoft Sentinel, Wazuh, Security Onion.
- Endpoint and network: CrowdStrike, SentinelOne, Snort, Suricata, firewalls, Wireshark.
- Skills used daily: log analysis, identity and access concepts, ticketing and report writing.

### 4. Hands-on projects and portfolio milestones
- Beginner: build a home lab with a Windows and a Linux virtual machine, generate your own logs and practise analyzing what happened
- Intermediate: deploy an open-source SIEM, write a few detection rules, and investigate sample alerts while documenting each invest
- Advanced: investigate a simulated phishing or malware incident end to end and write a professional incident report with a timeline,
```

10. ai_engineer.md

```
# AI Engineer (LLM Applications)

## Role guide
An AI engineer is a software engineer who builds applications powered by pre-trained AI models. They create chatbots, retrieval-augme

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: how LLMs work at a practical level (tokens, context windows, temperature, embeddings), APIs and JSON, basic machine
- Programming: Python, plus software engineering essentials such as testing, version control and API design.
- Development environment: Git and GitHub, VS Code, Jupyter, Docker, and secure handling of API keys.
- Switching from another field: developers have the biggest head start. A rough guide from one roadmap is 2 to 3 months for software

### 2. Core and technical skill progression
- Level 1, fundamentals: calling LLM APIs, prompt engineering (templates, structured JSON output, testing and versioning prompts), a
- Level 2, core competencies: RAG (chunking, embeddings, vector databases, retrieval, reranking), orchestration with LangChain or Lla
- Level 3, advanced: agents that use tools and complete multi-step tasks (still an experimental area), orchestration, LLMOps (loggin
- Durable skills such as programming, system design, testing and deployment stay valuable even when specific tools change.

### 3. Practical tooling and ecosystem
- LLM APIs: OpenAI, Anthropic Claude, Google Gemini and open-source models.
- Frameworks: LangChain, LlamaIndex.
- Vector databases: FAISS, Chroma, Pinecone, Weaviate, pgvector.
- Evaluation and observability: RAGAS, TruLens, LangSmith.
- Interfaces and deployment: Streamlit, FastAPI, Docker, and cloud platforms such as AWS Bedrock or Azure AI Studio.

### 4. Hands-on projects and portfolio milestones
- Beginner: a chatbot with well-designed prompts, a resume parser that returns structured JSON.
- Intermediate: a PDF chatbot with RAG, a customer-support RAG assistant, and an AI app with a Streamlit interface.
- Advanced: a tool-using agent, a RAG system with hybrid search, reranking and measured evaluation, a text-to-SQL app with guardrail
```

11. devops_engineer.md

```
# DevOps Engineer

## Role guide
A DevOps engineer connects software development and operations. They automate how software is built, tested, deployed and monitored,

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: how software moves from code to production, networking basics (DNS, IP, SSH), and the Linux operating system.
- Programming: Bash for quick automation, and Python for DevOps scripting and working with APIs.
- Development environment: Git and GitHub (branching, merging, pull requests), the terminal, VS Code.
- Switching from another field: system administrators, developers and testers have a head start. People from non-IT backgrounds also

### 2. Core and technical skill progression
- Level 1, fundamentals: Linux and shell scripting, Git, the basics of one cloud platform (virtual machines, storage, IAM, networkin
- Level 2, core competencies: CI/CD pipelines with Jenkins, GitHub Actions or GitLab CI, Kubernetes fundamentals, Infrastructure as
- Level 3, advanced: Helm and GitOps with ArgoCD, DevSecOps (scanning in the pipeline, secrets management), observability (metrics,
- Guides give different timelines: some say 8 to 12 months to become job-ready, others plan for 24 months or more. The orders also d

### 3. Practical tooling and ecosystem
- Version control and CI/CD: Git, Jenkins, GitHub Actions, GitLab CI.
- Containers: Docker, Kubernetes, Helm.
- Infrastructure as Code: Terraform, Ansible, CloudFormation.
- GitOps: ArgoCD or Flux.
- Monitoring: Prometheus, Grafana, ELK stack.
- Cloud platforms: AWS, Azure, GCP.
- Certifications often named: Certified Kubernetes Administrator (CKA), HashiCorp Certified Terraform Associate, and associate-level

### 4. Hands-on projects and portfolio milestones
- Beginner: a dockerized multi-service app (frontend, API, database and cache) with Docker Compose.
- Intermediate: a full CI pipeline that builds, tests and pushes a Docker image automatically, and a microservices app deployed on K
- Advanced: auto-deploy on every merge to the main branch with GitHub Actions and ArgoCD, AWS infrastructure defined in Terraform, a
```

12. cloud_solutions_architect.md

```
# Cloud Solutions Architect

## Role guide
A cloud solutions architect designs scalable, secure and cost-effective cloud solutions that meet business needs. They translate bus

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: cloud fundamentals, networking, compute, storage and databases, security basics, and design principles such as scal
- Programming and scripting: Python, Bash or PowerShell, and reading YAML and JSON.
- Development environment: Git, a cloud account for practice, and diagramming tools for architecture drawings.
- Switching from another field: cloud engineers, DevOps engineers, network engineers and developers with several years of hands-on e

### 2. Core and technical skill progression
- Level 1, fundamentals: one cloud platform's core services and an entry-level certification (AWS Cloud Practitioner, Azure Fundamen
- Level 2, core competencies: associate-level skills to design, deploy and manage secure, scalable applications (AWS Solutions Archi
- Level 3, advanced: architecture patterns and system design, cost optimization, multi-cloud and hybrid strategies, migration planni

### 3. Practical tooling and ecosystem
- Platforms: AWS, Azure, Google Cloud, Oracle Cloud.
- Infrastructure as Code: Terraform, Pulumi, Ansible, CloudFormation.
- Design guidance: the Well-Architected frameworks of AWS, Azure, Google Cloud and Oracle Cloud.
- Security: IAM, key management, network controls.

### 4. Hands-on projects and portfolio milestones
- Beginner: design and deploy a three-tier web app on one cloud.
- Intermediate: an architecture diagram with a written justification for scalability, availability and cost, built with Infrastructu
- Advanced: a multi-region, highly available architecture with a migration plan, a cost estimate and a security and compliance revie
```

13. generative_ai_engineer.md

```
# Generative AI Engineer

## Role guide
A generative AI engineer develops, tunes and deploys systems that generate text, images, code or audio. The work centers on large lar

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: linear algebra, probability, machine learning and deep learning fundamentals, how transformers and attention work, e
- Programming: Python. Many roles also expect familiarity with GPU computing.
- Development environment: Git and GitHub, Docker, Jupyter, the command line, and access to GPUs through a cloud or a notebook servic
- Switching from another field: ML engineers and software engineers can move over by learning prompt engineering, RAG and fine-tuning

### 2. Core and technical skill progression
- Level 1, fundamentals: how LLMs work (tokens, context windows, temperature and top-p), prompt engineering, calling LLM APIs, and th
- Level 2, core competencies: RAG pipelines (ingestion, chunking strategies, embedding models, indexing, retrieval and re-ranking), f
- Level 3, advanced: fine-tuning open-source models with parameter-efficient methods (PEFT, LoRA, QLoRA), supervised fine-tuning, ins

### 3. Practical tooling and ecosystem
- Frameworks: LangChain, LlamaIndex, Hugging Face (Transformers, PEFT), PyTorch.
- Vector databases: FAISS, Chroma, Pinecone, Weaviate, Milvus, pgvector.
- LLM APIs: OpenAI, Anthropic, Cohere, Google Gemini, and open models such as LLaMA.
- Deployment and operations: FastAPI, Docker, Kubernetes, MLflow, GPU toolkits (CUDA), cloud platforms.

### 4. Hands-on projects and portfolio milestones
- Beginner: a prompt-based application and a simple RAG app over a set of PDFs.
- Intermediate: a RAG pipeline with hybrid search (dense vectors plus BM25) and reranking, and a text-to-SQL application with validat
- Advanced: a LoRA or QLoRA fine-tuning pipeline with evaluation and model serving, an LLM gateway with rate limiting and fallback ro
```

14. backend_developer.md

```
# Backend Developer (Java, Python and API developer paths)

## Role guide
A backend developer builds the server side of an application: the logic, databases and APIs the app runs on. This covers data storage

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: how the web works (HTTP, requests and responses, client and server), data structures and algorithms basics, how data
- Programming: one server-side language learned well. Python (Django, Flask, FastAPI), Java (Spring Boot) or JavaScript with Node.js
- Development environment: Git and GitHub, VS Code or an IDE such as IntelliJ, the command line, Postman or similar for testing APIs
- Switching from another field: testers, data analysts and frontend developers already know part of this. Start with one language and

### 2. Core and technical skill progression
- Level 1, fundamentals: language basics, a web framework, SQL (queries, joins, schema design), REST API design with routing, validat
- Level 2, core competencies: authentication and authorization (tokens), protection against injection and endpoint abuse, ORMs, testi
- Level 3, advanced: design for scale and reliability, microservices, message brokers such as Kafka, observability, handling producti
- Rough guide to experience: Level 1 matches 0 to 2 years of work, Level 2 about 2 to 5 years, Level 3 about 5 years and beyond.

### 3. Practical tooling and ecosystem
- Frameworks: Django, Flask or FastAPI; Spring Boot; Express.
- Databases: PostgreSQL or MySQL, MongoDB for NoSQL, Redis for caching.
- Cloud and DevOps: Docker, GitHub Actions or another CI service, a cloud platform (AWS, Azure or GCP), logging and monitoring tools
- API work: OpenAPI or Swagger documentation, rate limiting, API testing tools.

### 4. Hands-on projects and portfolio milestones
- Beginner: a notes or to-do API with a database, a URL shortener.
- Intermediate: a blog or learning management system with login, roles, tests and a live deployment, an e-commerce API with payments
- Advanced: a multi-service app with queues, caching and monitoring, deployed with containers and a CI pipeline, and a load-tested AI
Common mistakes: skipping databases, and collecting certificates instead of building working projects.
```

15. java_developer.md

```
# Java Developer

## Role guide
A Java developer builds backend and enterprise applications in Java, most often with the Spring Boot framework. Java has a large ecos

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: object-oriented programming, data structures and algorithms, how the web works (HTTP, requests and responses), how
- Programming: Core Java and SQL. Learning Java before Spring is the usual advice, because Spring depends heavily on classes, interfa
- Development environment: Git and GitHub, IntelliJ IDEA or Eclipse, Maven or Gradle, the Linux terminal, basic Docker.
- Switching from another field: start with Core Java and SQL, practise with small console programs, and then move to Spring Boot.

### 2. Core and technical skill progression
- Level 1, fundamentals: variables, control statements, arrays and strings, OOP (inheritance, polymorphism, encapsulation, abstracti
- Level 2, core competencies: Spring Boot (starters, controllers, services, repositories), Spring Data JPA and Hibernate, REST APIs,
- Level 3, advanced: microservices with Spring Cloud (gateway, config server), containerizing with Docker, messaging with Kafka, moni
- Rough guide to experience: Level 1 matches 0 to 2 years of work, Level 2 about 2 to 5 years, Level 3 about 5 years and beyond.

### 3. Practical tooling and ecosystem
- Frameworks: Spring Boot, Spring MVC, Spring Security, Hibernate. JSP and Servlets appear mainly in older applications.
- Databases: MySQL, PostgreSQL or Oracle for relational data, MongoDB for NoSQL, Redis for caching.
- DevOps and cloud: Docker, Jenkins or GitHub Actions, Kubernetes, and a cloud platform (AWS, Azure or GCP).
- Frontend pairing for full stack roles: React or Angular.

### 4. Hands-on projects and portfolio milestones
- Beginner: console programs such as a library or bank management system, and CRUD with JDBC.
- Intermediate: a Spring Boot REST API with JPA, MySQL and JWT login (notes app, e-commerce mini project, blog app), with tests. One
- Advanced: a microservices application with an API gateway, config server, Kafka and Docker, with monitoring and a CI pipeline.
```

16. ai_research_engineer.md

```
# AI Research Engineer

## Role guide
An AI research engineer works at the point where model research meets training infrastructure. The job mixes open-ended research prob

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: linear algebra, calculus, probability and optimization, deep learning theory, and how transformers and large languag
- Programming: strong Python with efficient, maintainable code. C++ or Rust is useful for some roles.
- Development environment: PyTorch (JAX or TensorFlow are alternatives), Git, Linux, the command line, and the habit of reading paper
- Switching from another field: ML engineers and software engineers with distributed systems experience are well placed. Strong imple

### 2. Core and technical skill progression
- Level 1, fundamentals: machine learning and deep learning basics, building and training models in PyTorch, reimplementing a paper f
- Level 2, core competencies: transformers and LLM training, efficient training loops and data loaders, mixed-precision training, GPU
- Level 3, advanced: distributed training (data, tensor, pipeline and expert parallelism), large-scale training frameworks such as FS

### 3. Practical tooling and ecosystem
- Frameworks: PyTorch, JAX, TensorFlow.
- Distributed training: FSDP, DeepSpeed, Megatron-LM, CUDA, NCCL, MPI.
- Systems: GPU clusters, profilers and tracing tools, experiment tracking, and large-scale data pipelines.

### 4. Hands-on projects and portfolio milestones
- Beginner: implement a classic model such as a small transformer in PyTorch.
- Intermediate: reproduce and extend a recent paper with documented ablations, and train a small language model with mixed precision
- Advanced: a multi-GPU distributed training run using FSDP or DeepSpeed, with a profiling report that shows the bottlenecks found an
```

17. software_engineer.md

```
# Software Engineer / Software Developer

## Role guide
Software developers design computer applications or programs. They usually start by finding out how users will use the software, ther

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: programming fundamentals, data structures and algorithms, how computers and operating systems work at a basic level,
- Programming: at least one general-purpose language such as Python, Java, JavaScript, C++ or C#, and SQL.
- Development environment: Git and GitHub, an IDE or editor, the command line, a debugger.
- Education: a computer science or related degree is the usual route, but formal education is not strictly required. Employers typica
- Switching from another field: skills transfer from testing, data, IT support and design. Choose a direction (frontend, backend, ful

### 2. Core and technical skill progression
- Level 1, fundamentals (junior, roughly 0 to 2 or 3 years): builds tightly defined features with guidance. Focus on clean code, vers
- Level 2, core competencies (mid-level, roughly 2 to 5 years): builds vaguely defined features alone and owns them end to end. Skil
- Level 3, advanced (senior, roughly 5+ years): solves known problems without being told how. The shift is from contributor to multip
- Years per level differ between sources and companies, and smaller companies often use fewer levels: junior, mid, senior and lead.

### 3. Practical tooling and ecosystem
- Languages and frameworks: depend on the path (see the frontend, backend, full stack and mobile notes).
- Databases: PostgreSQL or MySQL, and a NoSQL option such as MongoDB.
- DevOps: Docker, CI with GitHub Actions or similar, a cloud platform, logging and monitoring.
- Industry knowledge also helps: a developer in a bank benefits from understanding finance.

### 4. Hands-on projects and portfolio milestones
- Beginner: command-line tools, small apps, algorithm practice, and a first web app.
- Intermediate: an end-to-end application with a database, an API and tests, plus contributions to a shared codebase.
- Advanced: a system with design documents, scalability and monitoring, and mentoring or reviewing others' work.
```

18. nlp_engineer.md

```
# NLP Engineer

## Role guide
An NLP (natural language processing) engineer builds systems that understand and generate human language. Typical tasks are text cla

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: probability and statistics, basic linear algebra, machine learning fundamentals (regression, classification), and ba
- Programming: Python with pandas and NumPy for data handling and text preprocessing.
- Development environment: Jupyter or Google Colab for fast experiments, Git and GitHub, and access to a GPU for deep learning.
- Switching from another field: data scientists and software engineers can start from text preprocessing and classical models and the

### 2. Core and technical skill progression
- Level 1, fundamentals: cleaning raw text, tokenization, stemming and lemmatization, NLTK and spaCy, classical machine learning for
- Level 2, core competencies: word embeddings (Word2Vec, GloVe), deep learning for text (RNN and LSTM), PyTorch or TensorFlow, then t
- Level 3, advanced: fine-tuning large language models with parameter-efficient methods such as LoRA, retrieval-augmented generation
- A stepwise path is advised: preprocessing and classical machine learning, then embeddings and sequence models, then transformers a

### 3. Practical tooling and ecosystem
- NLP libraries: spaCy, NLTK, Hugging Face Transformers and Datasets.
- Deep learning: PyTorch, TensorFlow and Keras. Classical machine learning: scikit-learn.
- Retrieval and search: embeddings with vector databases such as FAISS.
- Serving and deployment: FastAPI, Docker, MLOps frameworks for monitoring and retraining.

### 4. Hands-on projects and portfolio milestones
- Beginner: sentiment analysis, text cleaning pipeline, spam detection.
- Intermediate: news classification, named entity recognition, a customer support chatbot.
- Advanced: a fine-tuned transformer for a domain task (compared across models such as BERT and RoBERTa), a summarization system bui
```

19. machine_learning_engineer.md

```
# Machine Learning Engineer

## Role guide
A machine learning engineer takes an ML solution from experiment to production. Building a model is only one part of the job: the enc

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: linear algebra, probability and statistics, calculus (gradients), the ML lifecycle (data, features, training, evalua
- Programming: Python and SQL, plus software engineering basics (functions, modules, testing, version control).
- Development environment: Git and GitHub, Linux and the command line, Jupyter, Docker. Fluency with one cloud provider (AWS, Azure
- Switching from another field: software engineers add ML fundamentals and model evaluation. Data scientists add deployment, testing

### 2. Core and technical skill progression
- Level 1, fundamentals: supervised and unsupervised learning, feature engineering, model evaluation (accuracy, precision, recall, R
- Level 2, core competencies: deep learning with PyTorch or TensorFlow, reproducible training runs, experiment tracking, data and mo
- Level 3, advanced: deploying on Kubernetes with autoscaling and high availability, model registries, monitoring and drift detectio

### 3. Practical tooling and ecosystem
- Libraries: NumPy, pandas, scikit-learn, XGBoost, PyTorch, TensorFlow, Optuna for tuning, joblib, pytest.
- MLOps tools: MLflow for experiment tracking and model registry, DVC for data versioning, FastAPI for serving, Docker and Kubernetes
- Cloud: AWS, Azure or GCP managed ML services.

### 4. Hands-on projects and portfolio milestones
- Beginner: an end-to-end project with a baseline model and a written evaluation.
- Intermediate: a forecasting model (ARIMA or a simple RNN), a recommendation system served through an API, and a model deployed as a
- Advanced: a full MLOps pipeline covering training, a model registry, CI/CD, monitoring and a safe rollout (for example canary depl
```

20. bi_and_reporting_analyst.md

```
# Business Intelligence Analyst and Reporting Analyst

## Role guide
A BI analyst turns raw data into reports, dashboards and performance views that support business decisions. They usually own the repo

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: KPIs and metric definitions, data modeling basics (tables, keys, relationships, star schema), data quality, basic st
- Programming: SQL. Python is a plus.
- Development environment: Excel, Power BI Desktop (or Tableau), a SQL client, a data dictionary for documentation.
- Switching from another field: finance, operations and sales professionals already know the business metrics. Add SQL and a BI tool

### 2. Core and technical skill progression
- Level 1, fundamentals: SQL and Excel, data cleaning with Power Query, profiling data and writing a data dictionary, building simple
- Level 2, core competencies: data modeling with star schemas (a skill many beginners postpone, and often the reason their first dash
- Level 3, advanced: performance optimization for large datasets, governed semantic models, data governance and metric standards, dat
- A common pitfall in interviews is describing what a dashboard shows without being able to explain the data structure behind it.

### 3. Practical tooling and ecosystem
- BI tools: Power BI (with Power Query and DAX), Tableau, Looker.
- Data: SQL, Excel, data warehouses (for example Snowflake, BigQuery, Azure Synapse).
- Certification often named: Microsoft PL-300, Power BI Data Analyst Associate.
- Plus points in job posts: Python and cloud-based data management.

### 4. Hands-on projects and portfolio milestones
- Beginner: a recurring Excel KPI report with a data dictionary.
- Intermediate: a Power BI sales dashboard built on a star schema with DAX measures, and a churn or retention report.
- Advanced: a governed Power BI model with documented metric definitions, a refresh schedule, user guidance and performance tuning. I
```

21. site_reliability_engineer.md

```
# Site Reliability Engineer (SRE)

## Role guide
A site reliability engineer applies software engineering to infrastructure and operations problems so that services stay reliable, s

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: reliability thinking, distributed systems basics, Linux, networking, and how production systems fail.
- Programming: strong skills in at least one language, commonly Python, Go or Java.
- Development environment: Git and GitHub, the terminal, a cloud account, and containers.
- Switching from another field: DevOps engineers, system administrators, platform engineers and infrastructure engineers are the usua

### 2. Core and technical skill progression
- Level 1, fundamentals: Linux, networking, scripting, cloud and container basics, and basic monitoring. Entry-level work includes ta
- Level 2, core competencies: service level indicators (SLIs), SLOs and error budgets, observability with metrics, logs and traces, 
- Level 3, advanced: running major incidents, blameless post-mortems that lead to follow-up actions, reliability reviews of system a

### 3. Practical tooling and ecosystem
- Observability: Prometheus, Grafana, OpenTelemetry, Datadog, New Relic, Splunk.
- Incident management: PagerDuty, Opsgenie, incident.io.
- Infrastructure: Kubernetes, Terraform, cloud platforms.
- Languages: Python, Go.
- Certification often named: Google Professional Cloud DevOps Engineer.

### 4. Hands-on projects and portfolio milestones
- Beginner: a monitoring dashboard for a sample app with Prometheus and Grafana.
- Intermediate: define SLIs and SLOs for a service, add burn-rate alerts, and write a runbook.
- Advanced: simulate an outage, lead the response, write a blameless post-mortem with action items, and automate recovery with a Git
```

22. soc_analyst.md

```
# SOC Analyst (Security Operations Center)

## Role guide
A SOC analyst is the frontline of an organization's defence. They monitor alerts from tools such as a SIEM, decide which are real thr

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: networking fundamentals, how Windows and Linux log activity, and security concepts such as common attack types, aut
- Programming: basic scripting in Python or Bash helps with log analysis and automation.
- Development environment: a virtual lab, the Linux command line, a SIEM you can practise on, and a note-taking habit for investigat
- Switching from another field: with experience in help desk, system administration or networking, one guide estimates about 6 to 12

### 2. Core and technical skill progression
- Level 1, fundamentals: networking, operating systems and security concepts, then SIEM basics, such as searching and filtering logs
- Level 2, core competencies: SIEM proficiency, alert triage, investigating suspicious activity across logs, endpoint detection tools
- Level 3, advanced: detection engineering (writing and tuning alerts), proactive threat hunting, automation of repetitive work with

### 3. Practical tooling and ecosystem
- SIEM and logging: Splunk, QRadar, Microsoft Sentinel, Wazuh.
- Endpoint and network: EDR tools, intrusion detection (Snort, Suricata), Wireshark, packet and log analysis.
- Workflow: ticketing systems and playbooks, SOAR platforms for automation.

### 4. Hands-on projects and portfolio milestones
- Beginner: analyze sample logs and explain what happened.
- Intermediate: investigate sample alerts in a lab SIEM and write case notes for each, including how you decided it was real or a fa
- Advanced: write detection rules for common attacker behavior, tune them to cut false positives, write playbooks, and produce an in
```

23. frontend_developer.md

```
# Frontend Developer

## Role guide
A frontend developer builds the part of a website or web application that users see and interact with: pages, menus, forms and anima

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: how the web works (client and server, HTTP, browsers, URLs), basic design and usability ideas, accessibility basics,
- Programming: HTML, CSS and JavaScript. These three are the base of all frontend work.
- Development environment: Git and GitHub, VS Code, browser developer tools, basic command line, npm.
- Switching from another field: the same foundations apply. Your earlier domain knowledge (finance, retail, healthcare) helps you un

### 2. Core and technical skill progression
- Level 1, fundamentals: semantic HTML, forms, CSS layout with Flexbox and Grid, responsive and mobile-first design, JavaScript basi
- Level 2, core competencies: a framework (React is the most commonly listed; Vue and Angular are alternatives), components, hooks, 
- Level 3, advanced: frontend architecture for large apps, server-side rendering and frameworks such as Next.js, performance tuning,
- Rough guide to experience: Level 1 matches the first 0 to 2 years of work, Level 2 about 2 to 5 years, and Level 3 about 5 years a

### 3. Practical tooling and ecosystem
- Libraries and frameworks: React, Next.js, Tailwind CSS, a state library such as Redux or Zustand, Vue or Angular as alternatives.
- Build tools: npm, Vite or webpack.
- Data: frontend developers mostly consume REST or GraphQL APIs and use browser storage. Firebase is a common easy backend for proje
- Deployment and quality: Netlify or Vercel for hosting, GitHub Actions for CI, Lighthouse for performance checks, and an error trac

### 4. Hands-on projects and portfolio milestones
- Beginner: personal portfolio, to-do app, landing page clone, weather app that calls a public API.
- Intermediate: a React app with routing, login screens and API calls, such as a blog, movie search or e-commerce interface, with te
- Advanced: a production-style app with Next.js, TypeScript, a design system, performance and accessibility reports, and a CI pipeli
```

24. data_engineer.md

```
# Data Engineer

## Role guide
A data engineer builds and maintains the systems that move, store and prepare data so analysts, data scientists and applications can

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: computer science fundamentals, how databases store and query data, data modeling, the difference between a database,
- Programming: Python and SQL (Java or Scala for some roles), and Linux.
- Development environment: Git and GitHub, the command line, Docker basics, VS Code.
- Switching from another field: analysts add Python and orchestration, software engineers add data modeling and warehousing, and clo

### 2. Core and technical skill progression
- Level 1, fundamentals: Python for data processing, advanced SQL (joins, window functions, optimization), relational databases, data
- Level 2, core competencies: data warehousing and ELT (loading raw data first, then transforming inside the warehouse), dbt for SQL
- Level 3, advanced: Spark and PySpark for large-scale processing, Kafka and streaming pipelines, data lakes and lakehouses, CI/CD a

### 3. Practical tooling and ecosystem
- Orchestration and transformation: Apache Airflow, dbt, AWS Glue, Azure Data Factory.
- Processing and streaming: Apache Spark, Kafka.
- Warehouses and storage: Snowflake, BigQuery, Redshift, S3, GCS, Azure Blob, PostgreSQL.
- Cloud and DevOps: AWS, GCP or Azure, Docker, CI pipelines.
- Certifications often named: Google Cloud Professional Data Engineer and Azure Data Engineer Associate.

### 4. Hands-on projects and portfolio milestones
- Beginner: an ETL script that loads data from an API into PostgreSQL.
- Intermediate: a warehouse pipeline with Airflow and dbt, with data quality tests, for example a sales analytics pipeline.
- Advanced: a real-time pipeline with Kafka and Spark, or a complete cloud data platform. Show substantial SQL and Python, not only 
```

25. platform_engineer.md

```
# Platform Engineer

## Role guide
A platform engineer builds the internal platform that lets product teams ship software quickly and safely: tooling, environments and

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: Linux, networking, cloud fundamentals, how developers work (build, test, deploy, operate), and product thinking.
- Programming: Python, Go or TypeScript and JavaScript for internal tools and portals.
- Development environment: Git and GitHub, Docker, the terminal, a cloud account.
- Switching from another field: DevOps engineers, SREs and senior developers are the usual entrants.

### 2. Core and technical skill progression
- Level 1, fundamentals: the DevOps basics: Linux, Git, Docker, CI/CD pipelines and one cloud platform.
- Level 2, core competencies: Kubernetes and the wider CNCF ecosystem, Infrastructure as Code with Terraform, GitOps with ArgoCD or F
- Level 3, advanced: building an internal developer platform (IDP) with a developer portal such as Backstage, including custom plugi

### 3. Practical tooling and ecosystem
- Developer portals: Backstage, Port.
- Delivery: ArgoCD, Flux, Argo Rollouts, Helm, GitHub Actions.
- Infrastructure: Kubernetes, Terraform, Crossplane, managed Kubernetes services (EKS, GKE).
- Observability: Prometheus, Grafana.

### 4. Hands-on projects and portfolio milestones
- Beginner: a standard Dockerfile and CI template that any sample service can reuse.
- Intermediate: GitOps deployment of several services with ArgoCD from a Git repository.
- Advanced: a small Backstage portal with a software template that creates a new service with CI/CD, deployment and monitoring automa
```

26. python_developer.md

```
# Python Developer

## Role guide
A Python developer builds software in Python: web applications and APIs, automation scripts and data tools. On the web side the three

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: programming basics, object-oriented programming, data structures, how the web works (HTTP, requests and responses),
- Programming: Python and SQL.
- Development environment: Git and GitHub, VS Code or PyCharm, virtual environments and pip, the command line.
- Switching from another field: Python is a common first language, so start with the basics, build scripts that solve a real problem,

### 2. Core and technical skill progression
- Level 1, fundamentals: syntax, data types, functions, files and exceptions, object-oriented Python, modules and the standard libra
- Level 2, core competencies: a web framework (one guide suggests Flask basics first, then Django or FastAPI), REST APIs, routing, v
- Level 3, advanced: asynchronous programming and the difference between WSGI (Django, Flask) and ASGI (FastAPI), background tasks,
- Rough guide to experience: Level 1 matches 0 to 2 years of work, Level 2 about 2 to 5 years, Level 3 about 5 years and beyond.

### 3. Practical tooling and ecosystem
- Frameworks: Django with Django REST Framework, Flask, FastAPI.
- Databases: PostgreSQL or MySQL, SQLAlchemy, Redis for caching.
- DevOps and cloud: Docker, GitHub Actions, a cloud platform, and Nginx for production serving.
- Data libraries when needed: pandas, NumPy.
- Choosing a framework: Flask for small apps and microservices, Django for full web apps, FastAPI for new, high-performance APIs.

### 4. Hands-on projects and portfolio milestones
- Beginner: command-line tools, automation scripts, a small Flask app.
- Intermediate: a blog or learning management system with login, a database and tests, plus a REST API with FastAPI or Django REST F
- Advanced: a deployed multi-service application with background jobs, caching, containers and CI, with a short architecture write-u
```

27. computer_vision_engineer.md

```
# Computer Vision Engineer

## Role guide
A computer vision engineer builds systems that let machines interpret images and video. Typical tasks are image classification, obje

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: linear algebra, basic calculus and probability, how digital images are represented, and machine learning fundamenta
- Programming: Python with NumPy. C++ is useful for performance-critical roles such as embedded or real-time systems.
- Development environment: Git and GitHub, Jupyter or Colab, OpenCV, and GPU access (many free notebook services offer it).
- Switching from another field: software developers and data scientists can add computer vision by learning image processing and a d

### 2. Core and technical skill progression
- Level 1, fundamentals: image input and output, filters and transformations, drawing, contours, edge and corner detection with Open
- Level 2, core competencies: deep learning with PyTorch or TensorFlow (choose one), convolutional neural networks, image classifica
- Level 3, advanced: multi-object tracking (SORT, DeepSORT, ByteTrack), 3D vision, OCR, video AI, vision transformers and foundation

### 3. Practical tooling and ecosystem
- Libraries: OpenCV, PyTorch, TensorFlow, scikit-image, Pillow.
- Models: the YOLO family, Faster R-CNN, U-Net, Mask R-CNN.
- Deployment: ONNX and quantization for speed, Docker, FastAPI, GPU toolkits from NVIDIA, edge devices.
- Learn PyTorch well and practise implementing research papers.

### 4. Hands-on projects and portfolio milestones
- Beginner: edge and contour detection tool, a face mask detection system, lane detection from video.
- Intermediate: a CNN image classifier with transfer learning, and an object detection project with a custom dataset.
- Advanced: a segmentation or tracking pipeline, a real-time model optimized for an edge device, and an end-to-end system deployed t
```

28. full_stack_developer (1).md

```
# Full Stack Developer

## Role guide
A full stack developer works on both sides of a web application: the frontend users see and the backend that handles data and logic.

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: how the web works, client and server, HTTP and REST, how databases store data, basic security ideas such as passwor
- Programming: HTML, CSS and JavaScript, plus one backend language. Guides recommend choosing one stack and sticking with it: Java w
- Development environment: Git and GitHub, VS Code, browser developer tools, the command line, Postman.
- Switching from another field: frontend or backend developers add the missing half. Newcomers follow the whole order and keep to on

### 2. Core and technical skill progression
- Level 1, fundamentals: frontend basics (HTML, CSS, JavaScript), a frontend framework such as React, backend basics, SQL, and build
- Level 2, core competencies: connecting both sides through REST APIs, authentication and authorization, database design (SQL and No
- Level 3, advanced: system design, performance and caching, CI/CD, containers, cloud services, security, monitoring, and making tec
- Rough guide to experience: Level 1 matches 0 to 2 years of work, Level 2 about 2 to 5 years, and Level 3 about 5 years and beyond.

### 3. Practical tooling and ecosystem
- Frontend: React, Tailwind CSS, Vite or Next.js.
- Backend: Node.js with Express, Django, Flask or Spring Boot.
- Databases: PostgreSQL or MySQL, MongoDB, Redis for caching.
- Cloud and DevOps: Netlify or Vercel for the frontend, Render or Railway for the backend (free options named in guides), Docker, Gi

### 4. Hands-on projects and portfolio milestones
- Beginner: to-do or quiz app, a personal portfolio, a small API with a simple page that uses it.
- Intermediate: a notes or blog app and a mini e-commerce site with login, a database and a live deployment.
- Advanced: an application with roles, payments sandbox, background jobs, tests, containers and a CI pipeline, with a clear architec
Name your stack clearly on your resume, for example "React with Node.js and PostgreSQL".
```

29. data_analyst.md

```
# Data Analyst

## Role guide
A data analyst collects, cleans and analyzes data to support business decisions. Day to day, this means pulling data from databases v

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: basic statistics (mean, median, distributions, variance, correlation, probability), understanding the business probl
- Programming: SQL first, then Python later. Many guides say you do not need to learn every tool before applying.
- Development environment: Excel or Google Sheets, a SQL client, Jupyter Notebook, Git basics.
- Switching from another field: people from finance, operations or sales already know the business side. Add SQL, Excel and one BI to

### 2. Core and technical skill progression
- Level 1, fundamentals: Excel (formulas, lookups, pivot tables, cleaning, basic charts), SQL (SELECT, filters, joins, GROUP BY, subq
- Level 2, core competencies: a BI tool (knowing either Power BI or Tableau is enough), including data modeling, measures, KPIs and d
- Level 3, advanced: automating reports with Python, advanced Power Query and DAX, A/B test analysis, working with cloud data platfor
- Timelines in guides vary. One 6-month plan covers Excel and statistics, then SQL, then Power BI or Tableau, then Python.

### 3. Practical tooling and ecosystem
- Spreadsheets and SQL: Excel, Google Sheets, MySQL, PostgreSQL, SQL Server.
- BI tools: Power BI, Tableau, Looker.
- Python libraries: pandas, NumPy, Matplotlib, Seaborn.
- Platforms and certificates: cloud warehouses such as BigQuery or Snowflake, and certificates such as Microsoft PL-300 (Power BI), (

### 4. Hands-on projects and portfolio milestones
- Beginner: an Excel sales dashboard, SQL practice on a public dataset.
- Intermediate: a Power BI or Tableau dashboard with KPIs, and a Python exploratory analysis on a Kaggle dataset.
- Advanced: an end-to-end project from raw data through SQL to a dashboard and a written recommendation, an A/B test analysis, and a
```

30. data_scientist.md

```
# Data Scientist

## Role guide
A data scientist combines coding, statistics and problem solving to get insight and value out of data. Responsibilities include build

## Skill roadmap

### 1. Prerequisites and foundations
- Core concepts: probability and statistics (distributions, Bayes' theorem, hypothesis testing), linear algebra (vectors and matrices
- Programming: Python and SQL.
- Development environment: Jupyter Notebook, VS Code, Git and GitHub, the command line.
- Switching from another field: one guide advises learning data analytics skills first (SQL, pandas, basic statistics) and then build

### 2. Core and technical skill progression
- Level 1, fundamentals: Python with NumPy and pandas, SQL, data cleaning (missing values, duplicates, encoding, scaling), visualizat
- Level 2, core competencies: machine learning with scikit-learn. Supervised learning (linear and logistic regression, decision trees
- Level 3, advanced: deep learning when the problem needs it (for example images, text or very large data), model deployment (APIs, D

### 3. Practical tooling and ecosystem
- Libraries: NumPy, pandas, SciPy, scikit-learn, XGBoost, TensorFlow and PyTorch, Matplotlib, Seaborn, Plotly.
- Databases: SQL databases, plus data warehouses for larger data.
- Cloud and MLOps: AWS, GCP or Azure, Docker, and an experiment tracking tool such as MLflow.

### 4. Hands-on projects and portfolio milestones
- Beginner: an exploratory analysis notebook on a Kaggle dataset.
- Intermediate: a house price predictor (regression) and a customer churn classifier (random forest or XGBoost) with proper evaluati
- Advanced: an end-to-end deployed model, such as an image classifier or recommendation API with monitoring, and a write-up of the bu
```

31. analytics_engineer.md

Analytics Engineer

Role guide
An analytics engineer turns raw or messy business data into trusted analytical models. The role sits between data analyst and data e

Skill roadmap

1. Prerequisites and foundations
- Core concepts: data warehousing concepts, business context and KPI definitions, and how data analysts use data.
- Programming: SQL mastery first. Python helps with APIs, orchestration and testing, but most modeling work is still SQL.
- Development environment: Git and GitHub, the command line, VS Code, and a warehouse console.
- Switching from another field: data analysts and BI developers have strong transferable skills (SQL, dashboards, KPI definitions). W

2. Core and technical skill progression
- Level 1, fundamentals: advanced SQL, understanding the grain of a table, dimensional modeling (Kimball style), and warehouse basics
- Level 2, core competencies: dbt (models, sources, Jinja, tests, documentation), version control, building tested and documented mo
- Level 3, advanced: semantic layers, data quality and observability, CI/CD for data models, handling schema changes and data contra
- A suggested sequence in guides: SQL, then data modeling, then tested and documented projects.

3. Practical tooling and ecosystem
- Transformation: dbt and SQL (primary).
- Warehouses: Snowflake, BigQuery, Databricks SQL.
- BI: Looker, Power BI, Tableau.
- Quality and orchestration: dbt tests, data quality tools, Airflow.
- Certifications named in guides: dbt Fundamentals and dbt Analytics Engineering.

4. Hands-on projects and portfolio milestones
- Beginner: SQL models on a sample warehouse dataset.
- Intermediate: an e-commerce dbt project with dimensional models for orders, customers and products, with tests and documentation.
- Advanced: a data quality framework with custom tests and monitoring, or a migration of a legacy SQL reporting layer into dbt with